STAGE 00 · ANALYSIS — PRE-BUILD

BOOKING → CONTAINER → CASH

The MIS Blueprint

A single system for OLIU (Kigali) and JIVAH (Dar es Salaam) — one company, two flags — covering how a mineral container is booked, moved, cleared and tipped through to how it is invoiced, paid and reconciled.

GROUP	CARGO
OLIU · JIVAH	Tantalite · Lithium · Cassiterite
MODULES	STAGE
Operations / Accounting	Analysis — pre-build

CONTENTS

How this document is organised

What follows works through the physical chain in the order a container actually travels it (§03), then the money chain from invoice to bank statement (§04), the reconciliation mechanism itself (§05), and the roles, data model, AI scope, architecture, open decisions and build order that follow from both (§06–§11).

00	Executive summary	The two failures, and the one rule
01	Company & scope	One group, two jurisdictions, two modules
02	Guiding principles	Seven constraints made explicit
03	Operations module	The physical journey, as a data trail
04	Accounting module	From a closed container to a bank statement
05	Ops ↔ Finance reconciliation	Blind recording, then compare and flag
06	Users & roles	Including the Rusumo handoff
07	Data model	One line per relationship
08	Where AI belongs	Now, later, and never
09	Architecture	Modular monolith — remove a feature, break nothing
10	Open decisions	Five questions only you can decide
11	Build roadmap	Five phases, each a usable system

SECTION 00

Executive summary

OLIU and JIVAH are one freight-forwarding and customs-clearance operation for mineral cargo — tantalite, lithium, cassiterite, tin — moving out of Rwanda under two flags. A booking is placed with a shipping line for a batch of containers; each container then takes one of two physical paths: *local*, trucked once to Magerwa and finished, or *transit*, trucked across the Rusumo border into Tanzania, cleared through TRA and police checkpoints on a fixed transit clock, stored at an inland depot, handled at port, and loaded under an ocean freight charge billed back to whoever made the booking. At nearly every one of those physical stops, money changes hands — a transport invoice, a clearing fee, a port charge, or a tip paid on the spot to keep the container moving. Today, all of it lands in one shared Excel file, traceable back to a container only by whoever happens to remember which line belongs to which shipment.

That structure produces two specific failures, not a vague "we need a system" problem. First: because OLIU pays Rwanda-side costs and JIVAH pays Tanzania-side costs for the same shipment, and a single bulk transport invoice often covers several containers at once, nobody can currently isolate what one container actually cost — which is exactly why 2 delayed containers out of a batch of 25 can't be priced apart from the 23 that moved on schedule. Second: because the same person who pays a cost in the field is usually the only source for what Finance later books as that cost, there is no independent check that the two figures agree — a gap that an honest transcription slip and a dishonest one would look identical inside.

THE MECHANISM THIS ANALYSIS IS BUILT AROUND

Every franc, shilling, dollar and tip resolves to one container — never a booking, a month, or a shared pool. Operations' record of what was paid is then compared, automatically and after the fact, against Finance's independent record of the same line, with any disagreement flagged rather than silently corrected. The module boundaries, the tipping ledger and the accrual-based reporting that follow in this document are all consequences of that one rule.

What follows works through the physical chain in the order a container actually travels it (§03), then the money chain from invoice to bank statement (§04), the reconciliation mechanism itself (§05), and the roles, data model, AI scope, architecture, open decisions and build order that follow from both (§06–§11).

SECTION 01

Company & scope

One operating group, two jurisdictions, one ledger of trust between them.

OLIU Logistics (Kigali, Rwanda) and JIVAH Logistics (Dar es Salaam, Tanzania) are the same business wearing two flags, handling two ends of one journey: OLIU moves the minerals from Rwanda through Tanzanian transit to the Dar es Salaam port, and JIVAH takes it from there — loading it onto a vessel and sending it on to the actual buyer, an overseas client in China. The two entities constantly owe each other money for costs paid on the other's behalf, and it's JIVAH, not OLIU, that carries the relationship with the buyer once the cargo reaches port.

The MIS is scoped to exactly two modules, on purpose:

Operations

Everything that happens to a container between the moment it's booked and the moment it's ready to invoice.

Accounting

Everything that happens to money between an approved cost and a bank statement.

Nothing else is in scope. No dispatch-planning, no HR beyond payroll-as-an-expense, no CRM. That restraint is what makes "advanced but simple" achievable.

SECTION 02

Guiding principles

Seven constraints you stated, made explicit — every design choice below traces back to one of these.

Principle	What it rules in / out
Modular independence	Tipping, invoicing, payroll, reports are separate modules with their own tables and screens. Removing “Office Expenses” tomorrow must not touch “Payment Approvals”. Rules out one giant shared form or a single mega-table for “all costs”.
Manual entry, always	No shipping-line API integration is assumed or required. A human types the booking number and container numbers in. The system’s job is to never let them retype the same number twice with different meanings.
Blind recording, then reconcile	Operations staff record what they paid/received without visibility into what Finance later books for the same line. The system compares the two trails and only surfaces a flag when they disagree — see §05.
Container is the unit of truth	Every cost, tip, and revenue line must resolve to one Booking № + Container №. A month’s total is a report, not a stored number — see §03.
No silent duplicates	A container number that already exists, a repeat cost/payment against the same container + category + beneficiary, or a new transport leg opened after a breakdown, is never just quietly resolved by whoever caused it — each reports straight to Superadmin. Rules out a local manager “handling it” on their own — see §03.8.
Cash timing ≠ profit	Shipping $30M this month and collecting it next month is not a $30M loss. Reporting is built around when a container closes, not when a bank credit lands — see §04.
Digitised, not complicated	Every status that today travels by phone call or WhatsApp group gets a screen and a notification instead. The bar for a good screen is: “does this replace a phone call?”

SECTION 03

Operations module

The physical journey of one container, turned into a data trail.

03.1 The physical chain

Reconstructed directly from your notes, this is the sequence a container actually travels — and the sequence Operations records it in. Each stage below is a distinct screen and a distinct set of fields, not a free-text note.

01 — Booking

Shipping line first — it decides what the booking № should look like — then the booking №, containers, and client company, plus an initial routing guess (local, transit, or both) that §03.2's leg-by-leg record can extend as the journey actually unfolds.

02 — Transport

Truck, transport company, agreed price, route — the first of what can be several separately hired legs; see §03.2.

03 — Field release

Magerwa / Dubai — release cost + clearing cost, entered and signed off by the manager.

04 — Rusumo office

Border crossing. Release status + cost recorded in full.

05 — Checkpoints

TRA, police, weighbridges between Rusumo and the ICD — logging each stop is optional; the tips paid along the way are always recorded (§03.3).

06 — ICD

Inland depot — warehousing when it applies.

07 — Port charges

Totalled per container at the port.

08 — Ocean freight

OFT — JIVAH's leg, billed on to the buyer in China.

09 — Handoff to Finance

Container closes for Operations; invoice, payment and reporting take over.

Every stage writes to the same container record — nothing is re-typed at the next stage. Tipping is deliberately not one of these boxes: it can happen at any stage, for any reason — see §03.3.

SHIPPING LINE FIRST, BECAUSE THE BOOKING № FORMAT DEPENDS ON IT

CMA CGM, PIL, Hapag-Lloyd, Maersk and MSC each issue booking numbers in their own shape — some pure digits, some with a letter prefix, different lengths — there's no single pattern across carriers the way ISO 6346 gives every container number the same structure. So the Booking screen asks for the shipping line before it asks for the booking №, and the field only knows what a "normal-looking" number is for that line once it's picked — the same kind of typo-catching your mockup already does with the ISO 6346 check digit on container numbers. The exact expected format per shipping line is a configuration list, not something to hard-code from outside — whoever enters bookings against these five lines every day already knows each one's shape better than a spec written without seeing them.

03.2 Transit vs. local — tags on a leg, not the whole journey

The two transport types aren't alternate whole-journey templates for a container — they're tags on individual legs, and a container's transport is a chain of one or more legs, each its own truck, its own transporter, its own cost:

- **A local leg** — a truck is hired, picks up the cargo, and delivers it to Magerwa, to Dubai, or another named destination. Some containers' chains end here for good — not every export runs through Tanzania.
- **A transit leg** — a separately hired truck picks up from wherever the previous leg ended and runs some or all of the remaining chain from §03.1: field release, Rusumo, checkpoints, ICD, port, ocean freight.

Most export containers are actually both, in sequence: a local leg from the mine or warehouse to Magerwa, then a transit leg — a different truck, hired separately — carrying it on from Magerwa across Rusumo to the Tanzanian port. The second hire doesn't get squeezed into the first leg's record; it's simply the next leg in the same chain.

It's just as safe the other way round, and just as included: one truck hired for the whole run, picking up the cargo, passing through Magerwa, and carrying straight on across Rusumo into Tanzania without ever changing vehicle. A leg isn't defined by whether it happens to cross into transit territory — it's defined by the hire itself: one truck, one company, one agreed cost. So that's still just one leg, not two. Magerwa becomes a stop along that single leg's route rather than a boundary between separate legs, and whoever logs the leg simply marks it Transit, the same way they'd mark any leg that crosses into Tanzania. A second leg only ever appears when a genuinely different truck actually takes over the cargo — the common pattern above, but never something the system forces onto a journey that didn't actually happen that way.

A truck breaking down mid-route works almost the same way, with one deliberate difference: the replacement truck doesn't wait for anyone's approval to leave — the same "no approval gate on the way out" rule as tipping (§03.3) applies here too. What needs Superadmin's sign-off is the *record*, not the truck: whoever logs the breakdown opens the new leg immediately so the cargo keeps moving, but that leg sits pending until Superadmin confirms it, the same gate §03.8 uses for a duplicate container or a repeat payment. Inserting a new leg into an existing container's chain is exactly the kind of structural change that could otherwise hide a padded cost behind a genuine breakdown, so

the chain only grows with independent sign-off, even though nothing on the road stops for it. Confirmed or not, the original leg is never deleted or edited to make room for the new one — it stays exactly as it was, closed out with whatever cost it already incurred, so the container's history honestly shows two trucks and two costs, never one silently overwriting the other.

LOCAL OR TRANSIT: A CHOICE, NOT AN INFERENCE

Whoever creates a Transport leg picks Local or Transit directly, the same way they pick the truck and the price — the system never tries to work it out itself by checking whether the route happens to cross Rusumo. That's simpler and more reliable: geography-guessing logic is one more thing that can get it wrong, and the person hiring the truck already knows exactly what kind of job they've booked. It's also consistent with §02's "manual entry, always" principle — nothing here is inferred that a human can just say directly.

CONFIRMED: MORE THAN ONE DESTINATION

Magerwa — Rwanda's original bonded warehouse, in Gikondo — and Dubai are both real local-leg destinations, which settles it: this stays a free-text or short-list field the person hiring the truck fills in themselves, never a fixed value. The same field also covers whatever other named destination shows up that we haven't heard about yet — nothing about the design assumes the list stops at two.

Tipping, allocation, delays

03.3 Tipping — high-volume, so it has to be the fastest screen in the system

You were explicit on two points: tipping matters as much as any invoiced cost, and it isn't limited to checkpoints — it can happen for any reason, at any point, and it happens often, sometimes more than once in the same hour on a single journey. A form that takes two minutes to fill in will simply stop getting used correctly, and the whole reconciliation control in §05 depends on it being used every time. The driver has no login to this system, so most of the time it's someone else — a dispatcher taking a phone call, an escort if one travels with the cargo — actually holding the phone that logs it. So a Tip entry links directly to the container, full stop, and the screen is designed around speed first, for whoever that turns out to be:

- **Three taps, not a form.** The booking is already loaded — remembered from the last entry, since the same truck is usually still on the same journey — so its containers are already sitting there with their status, exactly as in §03.4. Tap the container (or containers, if several clear the same stop together, in which case §03.4's split does the arithmetic), tap a reason, tap or key an amount. Nothing else is required to save. Switching to a new journey means picking its booking once; every tip after that stays three taps. If it's a dispatcher entering it from a phone call rather than someone standing at the checkpoint, it's the same flow — just made a few minutes later, at a desk instead of a roadblock.
- **Reasons are buttons, not a dropdown** — a short, editable list (checkpoint passage, weighing/iminzani, expediting, goodwill, other) that someone taps without reading a menu. It's a configuration list, not a code change, so it can grow as real reasons show up.
- **Everything else is optional and can follow later** — a stage tag if it happened at a named checkpoint, the iminzani figures, a photo of a note. None of them block the entry from being saved in the moment; they can be attached to the same record afterward from a calmer screen.
- **Works offline, for whoever is actually holding the app.** Checkpoints are rarely where signal is good, and if an escort or agent ever travels with the cargo and carries the phone, their entry saves locally and syncs when a connection appears.
- **No approval gate on the way out** — nobody can wait for a manager's sign-off at a roadblock, whether it's a driver phoning it in or an escort tapping it in directly. The control happens afterward, in reconciliation (§05), not before the cash leaves someone's hand.

CHECKPOINTS: OPTIONAL, NOT MANDATORY

Logging each TRA and police stop individually — arrival, deadline, departure — is in the system, but nobody is required to fill it in. Tracking a checkpoint accurately only works if someone logs a true timestamp there, and since the driver has no system access, that timestamp usually arrives several minutes late by phone relay — which is exactly why it stays optional rather than something the container's status depends on. Whoever's able to log it (a dispatcher who took the call, an escort with the app) can tag a tip to a checkpoint and get the benefit — an early-warning flag on that specific leg if it's cutting it close on the 8-hour window — but a container with no checkpoint data logged is not treated as missing anything; it simply relies on ICD and breakdown flagging alone (§03.6), the way every container does today. Nothing here is built to assume checkpoint data exists, so coverage can be patchy without breaking anything, per §02's first principle.

WHY THIS GETS ITS OWN SCREEN

Every other cost in §03.7 is entered once, calmly, by someone at a desk. Tipping is entered many times, standing up, under time pressure — treating it as a smaller "add cost" form instead of a shortcut of its own is the single most likely reason a real system would get abandoned in favour of cash-in-pocket and a WhatsApp message.

03.4 How money lands on the right container

This was your open question: "whatever can happen, amounts will be assigned to booking number per every container." The answer is that nothing is ever entered as a bare number — every cost, tip or revenue screen starts by asking for Booking № first, and the instant that's entered, its containers populate automatically underneath it, each one showing its current status. Nobody re-types or hunts for a container number from a blank list; the booking already knows what's in it, and the screen shows it. Two consequences follow:

- **Shared costs — and shared tips — split automatically.** A truck carrying 3 containers on one invoice is entered once, against the transport leg, and divided across those 3 container records. A $90 tip paid once when the same 3 containers clear a checkpoint together works exactly the same way — $30 against each by default, evenly, by weight, or manually overridden, but always divided, never left sitting at the booking level or dumped onto whichever one container happened to be entered first.
- **A booking total is a query, never a stored figure.** "What did booking BKG-26-0141 cost" is always the sum of its containers' cost lines at the moment you ask — so a correction to one container's clearing fee can never leave the booking total stale.

03.5 One booking, many trucks — each container keeps its own history

Twenty-five containers on one booking rarely ride on a single truck — one truck usually carries a container or two, so a 25-container booking can be split across 15 or more separately hired legs from day one, not only once a few run late. Two containers sharing a booking number can be on

completely different trucks, on different roads, hours apart, and that's the normal case, not an exception.

Because a Transport leg lists exactly which containers it's carrying (§03.7) and a cost or tip splits only across those containers (§03.4), each truck's problems stay with that truck. If one breaks down, only the containers riding on it get the delay flag, the replacement leg, and the extra cost from §03.2 and §03.6 — the other containers in the same booking, on a different truck, are simply unaffected; they were never part of that leg's chain to begin with. A booking number ties containers together for invoicing and reporting; it was never a promise that they'd travel, or run into trouble, together.

Your idea for releasing a booking follows from exactly that: when a booking's containers do clear together, don't make someone step through a release action for each one. The release screen opens on that same auto-populated, status-labelled list from §03.4 — every container the booking's shipping line confirms as having reached the release stage shows up already selected, because its own status already says so. It's not a blanket "select all" that relies on someone remembering which ones to uncheck: a container whose status shows it hasn't gone that far yet — say, one still sitting at Magerwa while the rest of the booking has moved on — simply isn't pre-selected in the first place, because the status sitting right next to its name already says it isn't there. One authorised sign-off confirms the pre-selected group in a single action; unchecking is only ever a correction to a wrong status, not the normal way of catching a straggler. The container still at Magerwa stays exactly where its status says it is, unreleased — exactly the outlier state §03.6's delay-flagging already expects.

The harder part of your question is what happens to those 2 afterward. They don't wait for their own booking to move again — they get picked up whenever a truck has room, and that's very often a truck already carrying containers booked under a completely different, later booking number. A container's booking number never changes to match whichever load it ends up riding with, for the same reason established above: a Transport leg was never limited to containers from one booking in the first place.

Exceptions, records, Superadmin

03.6 Delays and exceptions

Your example: 25 containers booked today, 2 run late. Today that makes cost-per-container hard to count because the delay's extra cost (demurrage, an extra day of storage, a re-booked truck) gets buried in the same pile as the other 23. In this design, a delay is an event on the container's own timeline, not a separate spreadsheet:

- A container that sits at ICD past a set number of days is flagged at risk automatically — this is the same mechanism the existing mockup already shows for demurrage there.
- Any cost entered while a container is in that flagged state is tagged as delay-related, so "cost of the 2 late containers" and "cost of the 23 on-time ones" can be reported separately without anyone doing manual detective work at month-end.
- A mechanical breakdown is the same mechanism from the other direction: it's what actually closes one transport leg early and opens the replacement in §03.2, and the moment it does, the container is flagged at risk and every cost from the replacement truck onward is tagged delay-related — automatically, from the same event, not from someone remembering to flag it separately.

03.7 Operations records, in outline

Record	Fields
Booking	shipping line (picked first — sets the expected booking № format), booking №, client, date, container list
Container	container №, size, cargo, warehouse (if a local leg applies), status, current stage — routing (local-only / local+transit) is derived from its legs, not stored
Transport leg	truck, company, agreed cost, containers carried, route, type (local/transit), sequence № in the container's chain, status (completed / ended early — breakdown, pending Superadmin approval)
Checkpoint event *optional*	type (TRA/police/Rusumo/iminzani), timestamp in, deadline, timestamp out — only exists when someone was able to log it
Tip entry	container(s) — split automatically if more than one, reason, amount, currency, paid by, stage tag (optional), iminzani figures (optional)
Cost entry	container, category, amount, currency, paid to, entered by, delay flag
Handoff record	container, closed-by, closed date → opens the Accounting side

03.8 No silent duplicates — straight to Superadmin

A container number is unique across the whole system, not just within one booking — the same discipline your existing mockup already applies with its ISO 6346 check-digit validation. Three situations get routed the same way — not all of them strictly "duplicates," but each one a moment where a bad line could otherwise slip in looking exactly like an honest one:

- **The same container number entered twice.** Whether it's a typo, a re-used number, or the same box genuinely being billed under two bookings, the second entry doesn't just get quietly rejected or fixed by whoever typed it — it's reported directly to Superadmin, with both entries shown side by side.
- **The same cost or payment entered twice** against the same container, category and beneficiary (§04.5) works the same way: the override isn't a note the local Branch Manager waves through on their own — it goes to Superadmin.
- **A new transport leg opened for a replacement truck after a breakdown** (§03.2) goes through the same gate: Superadmin approves before the chain grows, so a genuine breakdown and a fabricated one both get the same independent check.

The reason all three bypass the normal approval chain, rather than routing to whichever manager is closest, is the same reason §05 keeps Operations and Finance blind to each other: each one is either an honest mistake or an attempt to slip in an extra cost, and the person best placed to explain it away quietly is often the person who caused it. Superadmin is the one role with no branch to protect.

SECTION 04

Accounting module

From a closed container to a bank statement — and a profit figure you can trust.

04.1 Scope: everything that moves money — kept as two books

Accounting here means everything your notes describe under that word — invoices, payments, reports, loss and profit, tax filings, even bank charges — not only the costs that flow out of Operations. Payroll, office rent, utilities and each entity's own tax return sit in this module on the same footing as a container's clearing fee; your existing mockup's "Office Expenses" screen is already this, just not yet named as part of the same whole.

And because OLIU and JIVAH are two flags on one business, each keeps its own book — its own manager, its own accountant, its own bank accounts, its own tax filings, its own approval chain. Nothing in §04.2–§04.6 requires merging them: a cost is booked under whichever entity actually paid it. The two books only meet at the group consolidation your mockup's CEO Cockpit already shows — and, per §06.1 below, exactly where a cost gets booked is decided automatically by where the container physically was when it was incurred.

04.2 The money flow, as you described it

Two payment patterns already exist and both stay: (a) many tips and small field costs are paid directly, in cash, and the invoice or receipt is emailed in afterward to be recorded against a payment order; (b) larger invoices — shipping line, clearing agents — are paid after a payment order is raised internally, and the client-facing invoice goes out only once the container's own costs are fully covered. Both patterns are the same one entity in the system, a Payment Order, just triggered at different points: before payment (pattern b) or after the fact to formalise a cash payment already made (pattern a). Your existing mockup's approval chain — requested → approved → paid → locked in the audit trail — applies to both without modification.

Because the buyer relationship sits with JIVAH (§06.1), the client-facing invoice for a container is raised from JIVAH's book by default — even when OLIU carried most of the Rwanda-side cost. That's not a special case to design around; it's the same "cost is booked under whichever entity actually paid it" rule from §04.1, applied to revenue instead of cost.

04.3 A single "needs payment" queue — select it, don't type it, and don't see Operations' side of it

Your instruction: whatever in the company needs paying — a container's clearing fee, a tip waiting to be reimbursed, this month's office rent, a payroll run — should show up as something to pick, not something to type. Retyping the same container, category and beneficiary is exactly where a mismatch or a duplicate payment starts; selecting an existing line removes that risk at the source. But making it easier for the Accountant can never come at the cost of showing her anything Operations recorded — that would make cheating easier, not harder, so the queue treats two kinds of "needs payment" quite differently:

- **Container-linked items** — a clearing fee, a tip reimbursement, anything that started life as an Operations cost entry — show up as nothing more than a flagged container: "PCIU 9084297 has a

pending item." No category, no beneficiary, no amount. The Accountant selects the container off that list — by booking or container number, the same structural lookup §06 already allows her — which removes the one typo risk that actually matters, matching numbers correctly. She then fills in the category, beneficiary and amount herself, from whatever invoice or receipt she's actually holding. She sees that something is owed on that container; she never sees what Operations said it was.

- **Accounting-native items** — office rent, payroll, a recurring bill, a supplier invoice Finance received directly — carry no Operations trail to protect in the first place, so these show in full: category, amount, due date, everything. There's nothing to blind here, because Finance was always the only source that ever touched the line.

Selecting a container still kills the typo risk that used to cause mismatches; it just stops well short of ever putting Operations' figure — or even Operations' description of the cost — in front of her. That independence is the entire point of §05, and this queue was built around not weakening it for the sake of convenience.

Once a line is paid, it drops out of the queue either way — which is §04.5's double-payment control working from the other direction: there's no stray "type it again" path left open, because there's nothing left to select.

Profit, double-pay, three numbers

04.4 Profit belongs to the container, not the calendar

> THE PROBLEM, IN YOUR WORDS
>
> We can ship $30M this month and get paid next month — that's not a $30M loss, it's a timing gap.

The fix is to report two different things side by side, instead of collapsing them into one "this month's number":

Container P&L

Revenue and cost recognised against a container the moment it closes (handoff from Operations, invoice raised), regardless of whether the client has paid yet. This is the number that answers "did this shipment make money."

Cash position

What's actually in the bank today, and what's overdue from whom. This is the number that answers "can we make payroll this month." Your mockup's dashboard already separates these two views (money in/out cards vs. the aged-receivables panel) — we're formalising that split as the reporting rule everywhere, not just the dashboard.

A container only ever appears in one month's P&L — the month it closed — even if the invoice is collected three months later.

04.5 Stopping a container from being paid twice

Because every payment order and every cost entry must carry a container reference (§03.4), the system refuses to raise a second payment order for the same container + cost category + beneficiary combination on its own — it goes to Superadmin instead, exactly as §03.8 describes for a duplicate container number. The same four-eyes discipline your mockup already applies to changing a beneficiary's bank account extends naturally to "pay this exact line again," and lands with the same independent role rather than whichever manager happened to raise it.

04.6 The three numbers you asked for

Question	Where it comes from
How much did this container cost?	Sum of every Operations cost entry tagged to it (§03.7), converted to USD at each cost's own date — exactly as your mockup already does.

Question	Where it comes from
How much did it earn?	The invoice(s) raised against it, net of any credit notes — independent of whether it's been collected yet.
Has it shipped?	A status on the container itself (booked → departed → arrived → cleared → delivered), the same lifecycle your mockup's "steps" bar already visualises.

SECTION 05

Ops ↔ Finance reconciliation

The control that answers: "record everything in the back, so Finance doesn't know what it was, and compare — flag to Superadmin if it doesn't match."

This is the single most important control in the system, so it's worth stating precisely. Operations and Finance each keep their own record of what a cost line was — Operations because they were there when the money moved, Finance because they process the invoice or bank transaction. Neither is shown the other's figure while entering theirs. A background process then compares the two independent trails for every container:

OPERATIONS TRAIL

What the person in the field says was paid, entered at the point of payment.

COMPARE ⇄

FINANCE TRAIL

What the invoice, receipt or bank transaction says was paid, entered on processing.

ON MISMATCH

Superadmin is notified with both figures, both authors, and both timestamps — nothing is auto-corrected, and neither record is overwritten.

A few things this deliberately does not do, so it stays a control and not friction:

- It never blocks either side from entering their number — the flag is raised after the fact, not as a validation error mid-form.
- A match produces no notification at all — silence is the expected, healthy state, exactly like the audit trail's "nothing happened" default.
- The tolerance for a "match" (exact figure vs. a small currency-rounding band) is a configuration, not a hard rule — see §10.

SECTION 06

Users & roles

Structured like the flow itself. Your mockup already defines the Finance-side roles. Operations needs its own, matching who actually stands at each stage of §03.1.

Role	Booking	Transport / field	Tip trail	Costs (enter)	Invoices	Payment approval	Beneficiaries	Users / config
Booking Officer new	✓	—	—	—	—	—	—	—
Operations Staff	view	✓	✓	✓	—	—	—	—
Branch Manager	✓	✓	view	✓	✓	✓	request	—
Accountant	view	*blind*	*blind*	✓	✓	—	request	—
Cashier	view	*blind*	*blind*	—	—	pay only	—	—
Managing Director	view	*blind*	*blind*	view	view	✓	2nd approval	—
Superadmin	view	both trails	both trails	view	view	view	view	✓

Two additions to what the mockup already shows: a Booking Officer role (the person who actually types a new booking in — today this is implicit, not assigned) and the fact that Superadmin is the only role that sees both trails in §05 side by side; everyone else only ever sees their own.

The “Booking” column and the two trail columns mean different things, and the difference is the whole point of this table. Booking = view means a role can look up any booking or container by number and see what your notes describe — its status, its stage, who’s involved — freely, exactly like the Accountant recording a payment needs to pull up the right container by its booking and container number. Transport/field and Tip trail = blind means that same Accountant never sees what Operations actually recorded as paid during the journey — not the transport cost, not a tip, nothing — while entering their own figure for that same line. They can find the container; they cannot see its Operations-recorded money trail. Only §05’s automatic comparison ever puts the two figures side by side, and only Superadmin sees the result.

This isn’t caution for its own sake: if the Accountant could see what Operations logged, they — or anyone with access to their login — could simply copy that number into the Finance entry regardless of whether it’s true, and §05 would report a “match” every time by construction. Keeping the trails blind from each other is what makes a match mean something.

VISIBILITY, NOT JUST PERMISSION

A tick in this table isn't "can act, otherwise sees everything anyway" — every role is also scoped to what it can see. A JIVAH Accountant sees JIVAH's containers, costs and invoices, not OLIU's; an Operations Staff member in Kigali sees the containers still on the Rwanda side of the chain — once one crosses Rusumo, it moves out of their working list entirely, not just out of their edit rights.

06.1 The Rusumo handoff

Rusumo isn't only a stage in §03.1 — it's where control of the container physically changes hands. Before it, the container is Rwanda-side: OLIU's people move it, pay for it, and see it in their working list. The moment it clears Rusumo, Tanzania-side control starts — JIVAH's people take over the checkpoints, the ICD, the port, and it's now their working list it belongs in.

The container record itself never splits — it's still one container from booking to delivery — but it carries a responsible branch that flips from OLIU to JIVAH the moment that stage is marked complete. That one flip does three things at once: it moves the container out of Kigali's working list and into Dar's, it tells §04.1's two books which entity a cost from that point defaults into, and it's the same event that already produces the inter-company balance your mockup's CEO Cockpit shows — "JIVAH owes OLIU — clearance costs paid by Kigali on Dar containers."

Rusumo is the internal handoff between the two branches; the port is the external one. OLIU is never in direct contact with the buyer — the container only becomes JIVAH's responsibility, and JIVAH only starts dealing with the client in China, once it has already crossed into Tanzania. That's consistent with why the ocean freight charge in §03.1's stage 08 belongs to JIVAH, not OLIU.

SECTION 07

Data model, at a glance

How the pieces from §03 and §04 actually relate — kept intentionally simple, one line per relationship.

Relationship	Meaning
Booking 1 —— * Container	One booking, many containers.
Container * —— * Transport leg	An ordered, chained sequence — local and/or transit legs, one truck at a time; a breakdown ends one leg and opens the next.
Container 1 —— * Cost entry	Costs accumulate against one container; a shared cost — or tip — splits into several entries.
Container 0/1 —— * Checkpoint event	Optional — only present where someone was able to log it (§03.3); most legs will have none, and that's fine.
Container * —— * Tip entry	One payment can cover several containers at once — it splits the same way a shared cost does.
Tip entry * —— 0/1 Checkpoint event	Optional link, only when the tip happened at a named stop someone logged.
Container 1 —— * Invoice line	An invoice can bundle several containers for one client.
Invoice —— 1 Payment order (in)	Client payments received against an invoice.
Cost entry —— 1 Payment order (out)	Supplier/beneficiary payments made against a cost.
Cost entry ⇄ Finance record	The reconciliation pairing from §05 — a link, not a merge.

Nothing here requires re-architecting the mockup's existing Container, Invoice, Payment or Beneficiary screens — Operations adds new tables (Tip entry, Transport leg, and an optional Checkpoint event) that feed the same Cost entry the Finance side already reads from.

SECTION 08

Where AI genuinely belongs

Split into what's simple enough to build now and what should wait for real data — kept out of anything that authorises money either way.

08.1 Now — low complexity, worth having from day one

One thing needs no history of its own to start working, and touches nothing that authorises money — worth having from the start rather than as a "someday" feature:

Use	Why it's simple enough to do now
Plain-language reports	"What did we earn on Space Mining's containers this month" answered directly from the container P&L data in §04.4 — a convenience layer over reports that already exist, not a new source of truth.

08.2 Future — worth revisiting once there's real history, or a manual step becomes the bottleneck

None of these three need building now. For now, assume email keeps being read and typed in by people exactly as it is today. Worth designing the data model to leave room for later (it already does), not worth building before there's a real need or enough history to learn from:

Use	What it's waiting on
Reading emailed invoices, receipts & field photos	Only worth it once manual entry is actually the bottleneck. The same skill would also cover a breakdown photo attached to a replacement leg (§03.2) — reading it into a suggested reason for whoever approves the leg, rather than someone typing a description. Input like this suits AI well and needs no history to work; it's simply not needed yet, since people already handle it.
Reconciliation triage	Ranking flagged mismatches by how unusual they are (e.g. one beneficiary's tips consistently 3× its peers) needs enough historical flags to know what "unusual" looks like.
Delay risk on a route	Flagging a container as at-risk before it happens needs enough history of ICD dwell times and leg breakdowns by route to beat a manager's own instinct — plus whatever optional checkpoint data (§03.3) happens to exist, used opportunistically rather than relied on.

DELIBERATELY NOT USING AI FOR

Approving a payment, verifying a beneficiary's bank account, or deciding a reconciliation mismatch is "fine." Those stay deterministic and human-approved — AI's role throughout is to reduce typing and rank attention, never to authorise.

SECTION 09

Architecture

Built for "remove a feature, break nothing." Modular monolith, not microservices — the independence you asked for without the operational overhead a 12-person company doesn't need.

Each of the following is a self-contained module with its own tables and its own screens, talking to the others only through a small, explicit interface (an event like *container.closed*, or a lookup like "cost entries for container X") — never by reaching into another module's tables directly:

Identity & roles

users, roles, branch access — everything in §06

Operations

booking, container, transport, tipping — §03

Accounting

invoices, payment orders, beneficiaries, expenses — §04

Reconciliation engine

the compare-and-flag control in §05, reading both modules, owned by neither

Notifications

the bell, email, SMS — replaces the WhatsApp group

Audit trail

append-only, five-year retention, already proven in the mockup

Reporting

container P&L, cash position, aged receivables — reads from the others, owns nothing

Data import

the Excel-migration flow already designed in the mockup

Practically: turning "Office Expenses" off tomorrow removes one module and its screen from the sidebar — Payment Approvals, Reconciliation and everything else keep running untouched, because nothing else was ever allowed to depend on its internals.

SECTION 10

Open decisions — before we build

Not blockers to keep analysing — things only you can decide, each with a reasonable default noted.

Q1 · RECONCILIATION TOLERANCE

What size of gap between the Operations figure and the Finance figure is a rounding difference vs. a real flag?

Default: exact match required in the entered currency; a configurable small percentage band for currency-conversion drift only.

Q2 · SUPERADMIN

Who exactly is Superadmin day to day — the MD personally, or a trusted operations lead reporting to the MD?

This decides how many mismatch flags a real person can be expected to act on.

Default: MD, with a delegate role addable later without changing the model.

Q3 · BOOKING OFFICER

Who holds it today?

Right now booking entry isn't assigned to a named role.

Default: whoever currently types bookings into Excel becomes this role unchanged.

Q4 · HOSTING & CONNECTIVITY

Tip entries happen on the road, often with weak signal, for whoever ends up logging them.

Does the offline-and-sync design in §03.3 need to hold up for a field device, or is it only ever a dispatcher on a normal office connection? Q5 decides which.

Q5 · ESCORT WITH THE CARGO?

Does anyone travel with the cargo who could hold the app?

A clearing agent or escort with a phone turns a tip entry into something logged the moment it happens; without one, every entry is necessarily relayed by phone to a dispatcher a few minutes later, per §03.3.

Default assumed: nobody does — all field entries are phone-relayed. If an escort does travel with high-value mineral loads, that changes §03.3 for the better and is worth telling us.

SECTION 11

Build roadmap

Five phases, each one a usable system on its own — not a big-bang release.

01 **Operations core**

Booking, container, transport, cost entry — the backbone every later phase reads from.

02 **Tipping, with optional checkpoints**

Iminzani figures and the tip ledger — the part with no Excel equivalent today — plus the optional checkpoint-event log for whoever's able to use it, per §03.3.

03 **Accounting**

Largely the screens already proven in your mockup — invoices, payment approvals, beneficiaries — wired to real Operations data instead of sample rows.

04 **Reconciliation & reporting**

The Ops↔Finance compare-and-flag engine, plus container P&L and cash-position reporting.

05 **Migration & future AI**

Bring the Excel history across using the import flow already designed, then revisit reconciliation triage, delay-risk flagging, and AI email reading (§08.2) once there's enough real history — or once manual email handling actually becomes the bottleneck.